MACHINE LEARNING (23A31401T)

UNIT – I : Introduction to Machine Learning

Easy digital notes with diagrams for students

Topics in this unit

1. Evolution of ML, paradigms for ML
2. Learning by rote, by induction, reinforcement learning
3. Types of data and matching
4. Stages in machine learning (the full pipeline)
5. Data acquisition, feature engineering, data representation
6. Model selection, learning, evaluation, prediction
7. Search and learning, data sets

> * **Study tip:**
> Every topic has a small diagram. Learn the RED words first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

Introduction to Machine Learning: Evolution of Machine Learning, Paradigms for ML, Learning by Rote, Learning by Induction, Reinforcement Learning, **Types of Data**, Matching, Stages in Machine Learning, Data Acquisition, Feature Engineering, Data Representation, Model Selection, Model Learning, Model Evaluation, Model Prediction, Search and Learning, Data Sets.

Part I : Basics, Evolution and Paradigms

I.I Evolution of Machine Learning

Machine Learning (ML) = a branch of AI where a computer **learns from data** and improves with experience, without being told every rule.

- **Tom Mitchell (1997):** a program learns from experience **E** for task **T** with performance **P**, if its performance on T improves with E.
- Example: T = spam detection, E = emails marked spam / not spam, P = % of mails classified correctly.
- AI is the big idea, ML is a part of AI, and **Deep Learning (DL)** is a part of ML.

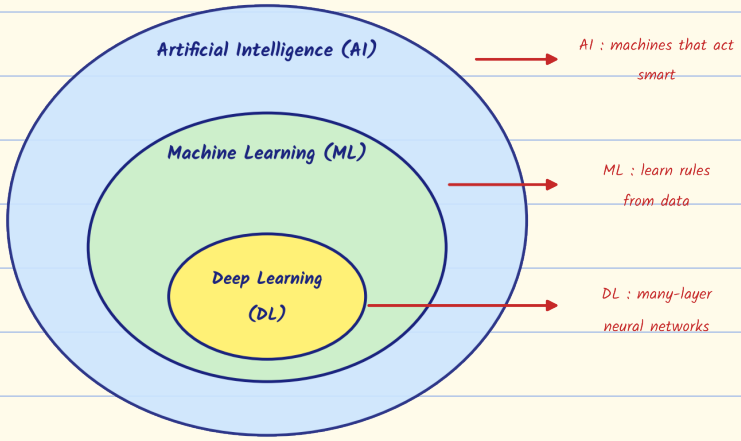


Fig I.I : AI, ML and DL (nested)

Traditional programming: human writes rules, computer applies them to data. **ML:** we give data and answers, computer finds the rules (the **model**).

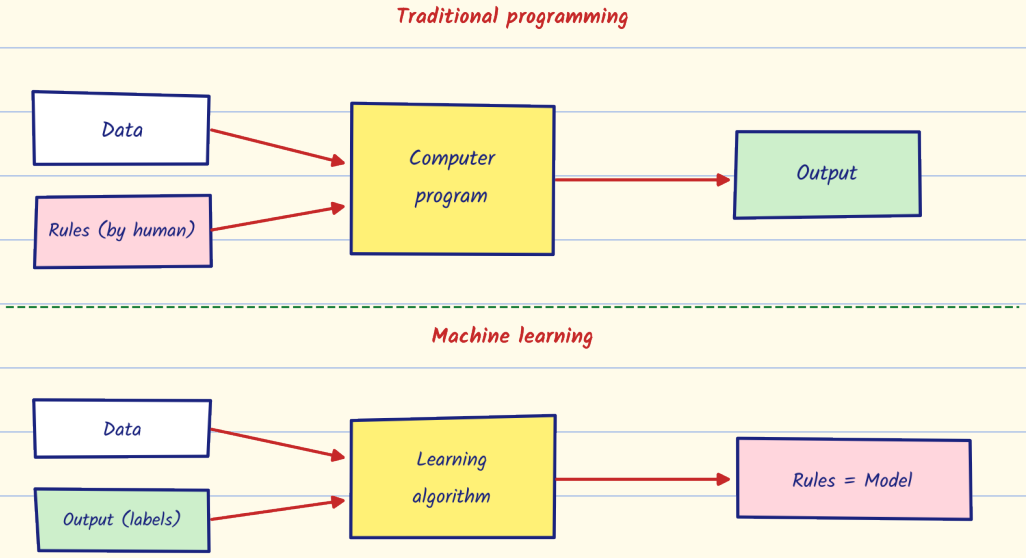


Fig 1.2 : Traditional programming vs machine learning

Short history of ML:

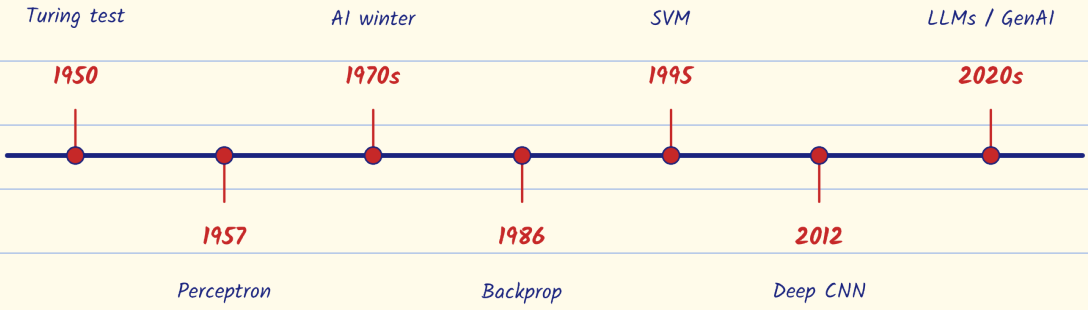


Fig 1.3 : Timeline of machine learning

Period	Key idea
1940s-50s	Neuron model (McCulloch-Pitts), Turing test (1950), Samuel checkers program - term "machine learning" (1959)
1957-60s	Rosenblatt Perceptron - first learning machine; nearest neighbour rule
1970s	Limits of perceptron found -> **AI winter**; rule-based expert systems
1980s-90s	Backpropagation (1986), decision trees, **SVM** (1995), statistical ML, Bayesian methods
2000s	Big data, ensembles (random forest, boosting), Hadoop, better CPUs
2012 onwards	**Deep learning** with GPUs (AlexNet), speech, vision; later Transformers and LLMs / generative AI

* **Remember:**

Three things made ML boom : **more data**, **faster hardware (GPU)** and **better algorithms**.

I.2 Paradigms for Machine Learning

A **paradigm** = a style of learning. It depends on what kind of feedback the learner gets.

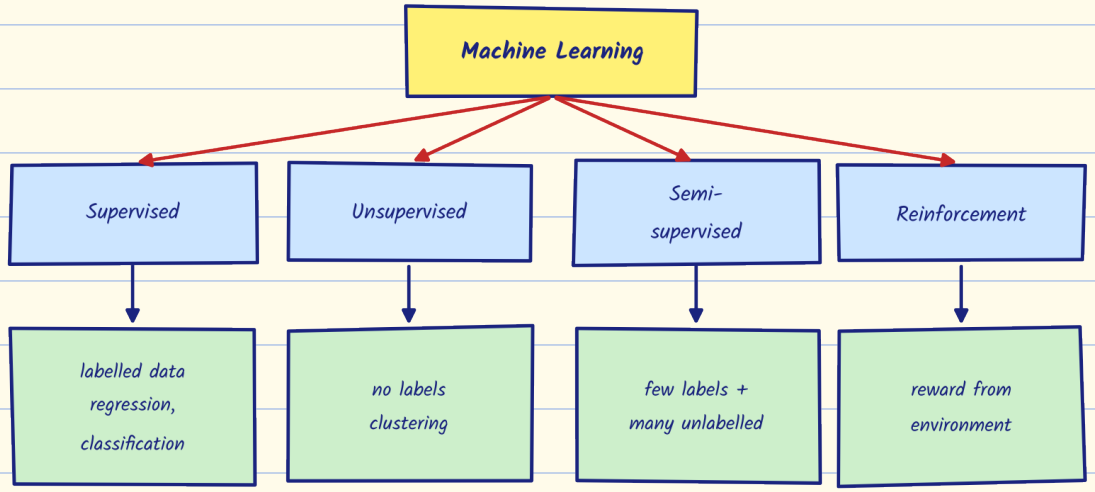


Fig I.4 : Paradigms (types) of machine learning

Paradigm	Data used	Goal	Example
Supervised	Inputs + correct labels	Learn map x -> y (classification, regression)	Spam filter, house price
Unsupervised	Inputs only, no labels	Find hidden structure (clustering, association)	Customer groups
Semi-supervised	Few labelled + many unlabelled	Use cheap unlabelled data to help	Photo tagging
Reinforcement	Reward / penalty signal	Learn best actions	Game playing, robots

- **Classification** - output is a class (spam / not spam). **Regression** - output is a number (price = 52.3 lakh).
- Learning can also be seen as **rote**, **by induction** and **by reinforcement** - explained next.

I.3 Learning by Rote

Rote learning = learning by **memorising**. The system simply stores the examples (a lookup table) and gives the stored answer when the same input comes again.

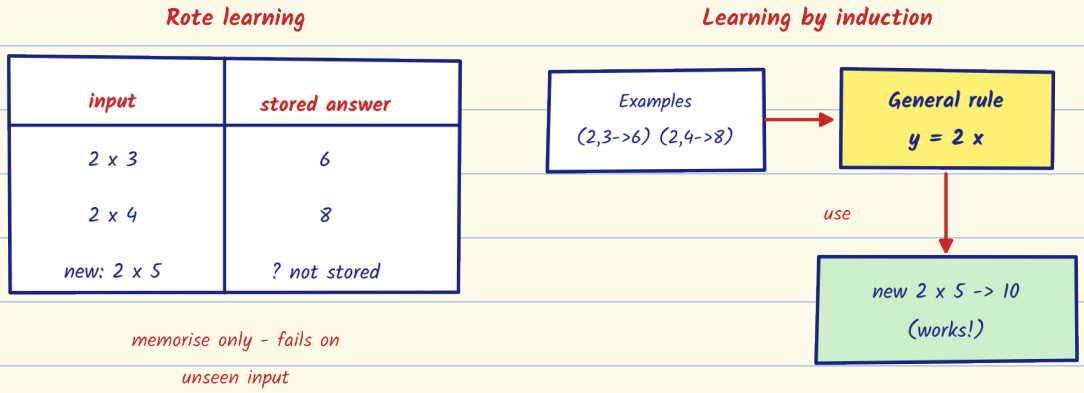


Fig 1.5 : Rote learning (lookup table) vs learning by induction

- No understanding, **no generalisation** - an unseen input has no answer.
- Very fast to recall, but memory grows with every new example.
- Example: Samuel checkers program stored board positions with their values; a student mugging up tables.
- In ML terms, a pure **look-up / caching** system (store input -> output).

1.4 Learning by Induction

Induction = going from **specific examples to a general rule**. The learner sees many examples and builds a hypothesis that also works for **new, unseen** data.

- Seen: sparrow flies, crow flies, eagle flies -> rule: **birds fly** (a general hypothesis).
- This is the **core of supervised learning** : training examples -> model -> predict on new data.
- Uses **inductive bias** = assumptions that help pick one rule among many possible rules.
- Induced rules may be wrong (penguins do not fly), so we must **test** them on fresh data.

Point	Rote learning	Induction
What is stored	Raw examples	General rule / model
Unseen input	Fails	Can predict
Generalisation	None	Yes (main aim)
Memory	Grows with data	Fixed size model

1.5 Reinforcement Learning

In **reinforcement learning (RL)** an **agent** interacts with an **environment**. It takes an **action**, and gets a new **state** and a **reward** (or penalty). It learns a **policy** that gives the maximum total reward.

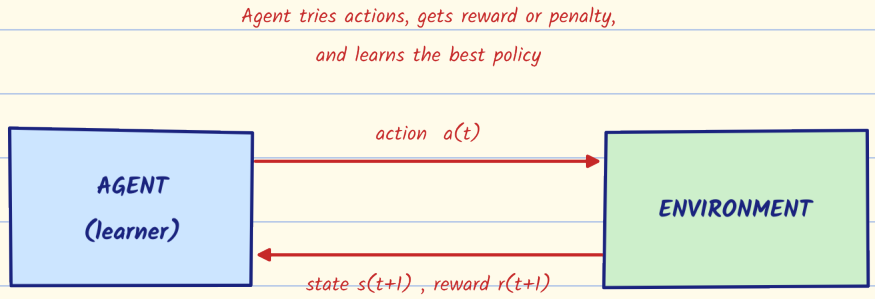


Fig 1.6 : Agent - environment loop in RL

- No labelled answers - only **trial and error** with delayed reward.
- Key terms: **agent, environment, state, action, reward, policy**.
- Balance of **exploration** (try new actions) and **exploitation** (use best known action).
- Examples: chess / Go (AlphaGo), self-driving car, robot walking, ad recommendation.

> * **Example:**
> A robot in a maze: wall hit = -1, reaching exit = +10. After many tries it learns the shortest path - nobody told it the path.

Part 2 : Types of Data and Matching

1.6 Types of Data

ML needs data, and the **type of data** decides which algorithm and pre-processing we use.

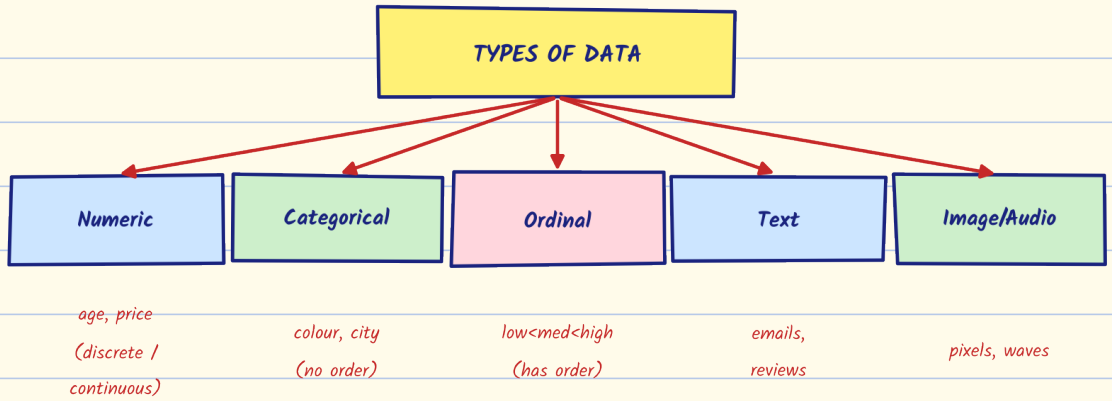


Fig 1.7 : Types of data

Type	Meaning	Example
Numeric (quantitative)	Numbers; **discrete** (counts) or **continuous** (measured)	Age 21, salary 35000, temp 36.6

Type	Meaning	Example
Categorical (nominal)	Labels without order	Gender, city, colour
Ordinal	Categories with a meaningful order	Grade A>B>C, rating 1-5
Binary	Only two values	Yes/No, 0/1
Text	Unstructured words	Reviews, tweets, emails
Image / audio / video	Pixel or signal values	X-ray, MNIST digits, speech
Time-series	Values in time order	Stock price, ECG, sensor log

- **Structured** data = rows and columns (database). **Unstructured** = text, images, audio. **Semi-structured** = JSON, XML.
- **Labelled** data has the correct output; **unlabelled** data does not.
- Categorical data must be **encoded** to numbers (e.g. one-hot) before most models.

I.7 Matching

Matching = comparing a **new pattern** with stored patterns or **prototypes** to find the closest one. The class of the best match is the answer.

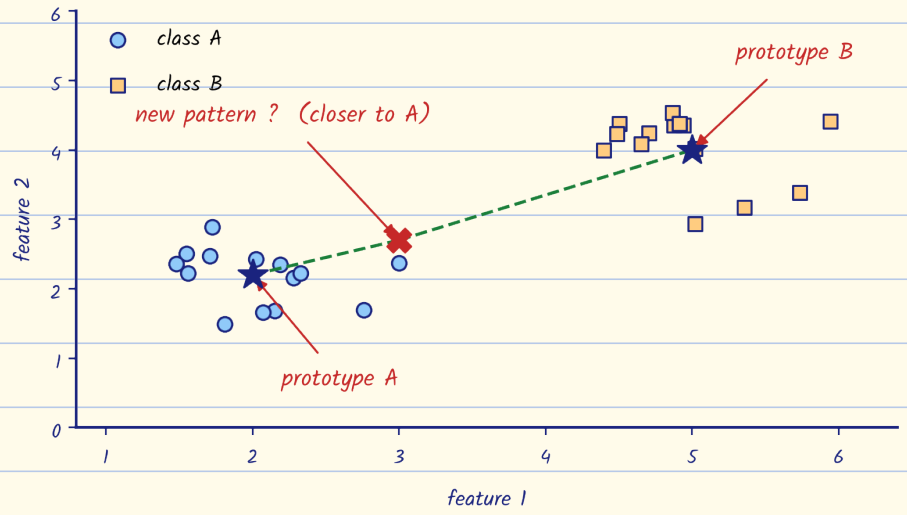


Fig I.8 : Matching a new pattern with class prototypes

- Needs a **similarity / distance measure** : Euclidean, Manhattan, cosine.
- Euclidean distance: d(a,b) = sqrt((a1-b1)^2 + (a2-b2)^2 + ...).
- **Exact matching** : identical (string, key lookup). **Approximate matching** : nearest / most similar.
- Used in **nearest neighbour** classifiers, template matching, face / fingerprint recognition, search engines.

- Matching can be against **every stored example** (k-NN) or only a **representative prototype / mean** of each class (nearest centroid).

* **Example:**
New point (3, 2.7). Distance to prototype A (2, 2.2) = sqrt(1 + 0.25) = 1.12. Distance to B (5, 4) = sqrt(4 + 1.69) = 2.39. Smaller distance -> class **A**.

Part 3 : Stages in Machine Learning

1.8 Stages in Machine Learning

Building an ML system is a **pipeline** of stages. Results of later stages often send us **back** to improve earlier ones.

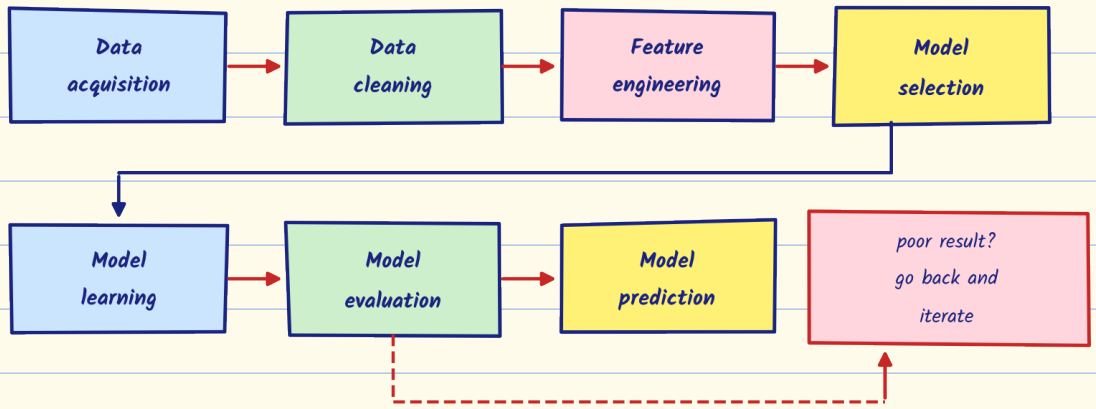


Fig 1.9 : Stages (pipeline) of a machine learning project

1. **Problem definition** - what to predict, what is success?
2. **Data acquisition** - collect data.
3. **Data preparation / cleaning** - missing values, noise, duplicates.
4. **Feature engineering** - build useful features.
5. **Model selection** - pick algorithm.
6. **Model learning (training)**.
7. **Model evaluation** - test on unseen data.
8. **Model prediction** (deployment) - use on new data, monitor.

1.9 Data Acquisition

Data acquisition = collecting the raw data needed to train the model. **Good data matters more than a fancy algorithm** (garbage in, garbage out).

- **Sources:** databases, sensors / IoT, web scraping, APIs, surveys, logs, public repositories (UCI, Kaggle).
- **Quality checks:** enough **quantity**, **relevance**, **accuracy**, balanced classes, no bias, legal / privacy rules.
- **Cleaning:** fill **missing values** (mean / median), remove **duplicates**, handle **outliers** and noise, fix wrong labels.
- **Pre-processing: normalisation** (scale to 0-1) : x_new = (x - min) / (max - min); **standardisation** : z = (x - mean) / std.

1.10 Feature Engineering

A **feature** = one measurable property of the data (age, pixel value, word count). **Feature engineering** = creating, choosing and transforming features so the model learns better.

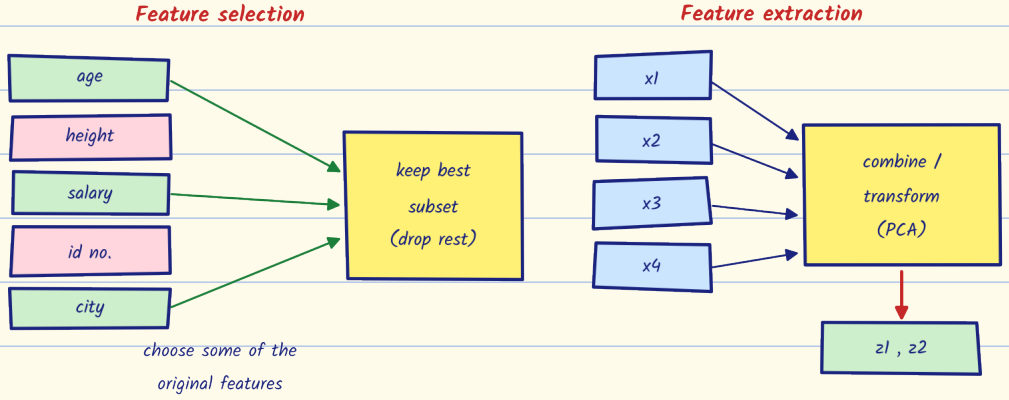


Fig 1.10 : Feature selection vs feature extraction

Technique	Idea	Example
Feature construction	Make new feature from old	BMI = weight / height^2; date -> day of week
Feature selection	Keep the useful subset, drop the rest	Drop id number; filter, wrapper, embedded methods
Feature extraction	Combine features into fewer new ones	PCA, LDA, TF-IDF for text
Encoding / scaling	Make features model-friendly	One-hot, label encoding, normalisation

- Benefits: **less over-fitting**, faster training, better accuracy, easier to understand.
- **Curse of dimensionality** : too many features need exponentially more data.

1.11 Data Representation

Computers need data as **numbers** in a fixed form. Each sample is written as a **feature vector**, and the whole data set as a **matrix**.

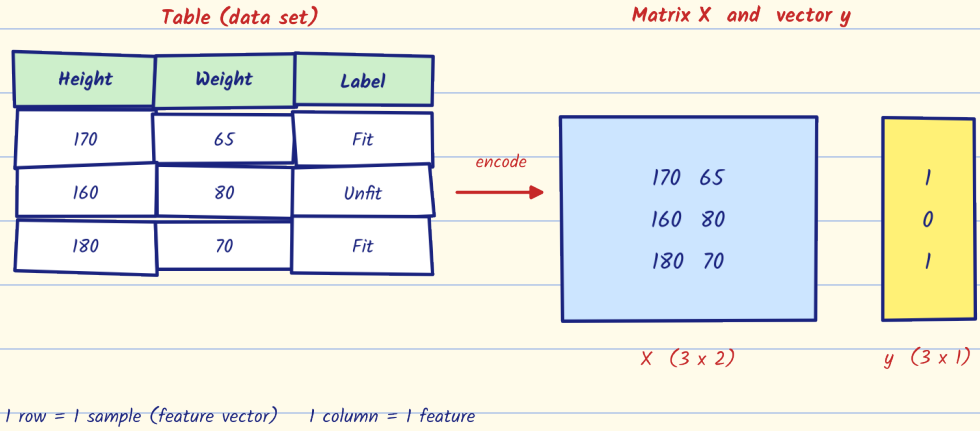


Fig 1.11 : Data set as a table, feature matrix X and label vector y

- Sample i = **feature vector** $x = (x1, x2, ..., xd)$. Data set = matrix X of size $n \times d$ (n samples, d features).
- Labels are stored in vector **y** (size n). Supervised data = pairs (x, y).
- **Images** -> pixel matrix / flattened vector. **Text** -> bag of words, TF-IDF, word embeddings. **Categories** -> one-hot vectors.
- Other representations: **graphs** (social network), **trees**, **sequences** (time-series), **logic rules**.

> * **Remember:**
> One-hot for colour {Red, Green, Blue}: Red = (1,0,0), Green = (0,1,0), Blue = (0,0,1). No false order is created.

1.12 Model Selection

Model selection = choosing the best **type of model** (and its settings) for the problem. A model is a function **h(x)** with **parameters**.

- Depends on: **type of problem** (classification / regression / clustering), size and type of data, need for interpretability, time and memory.
- Candidates: linear / logistic regression, k-NN, decision tree, SVM, naive Bayes, neural network.
- **Parameters** are learnt from data (weights). **Hyper-parameters** are set by us before training (k in k-NN, tree depth, learning rate) and tuned with **validation data / cross-validation**.

- **Occam's razor** : among models with similar accuracy, choose the **simpler** one.
- Model complexity balance: too simple -> **under-fit** (high bias); too complex -> **over-fit** (high variance).

1.13 Model Learning

Model learning (training) = using the training data to find the parameters that give the **least error**.

- Define a **loss (cost) function**, e.g. mean squared error: J = (1/n) x sum of (y - y_hat)^2.
- An **optimisation** method reduces the loss step by step - e.g. **gradient descent** : w = w - alpha x (slope of J).
- Loop: predict -> measure loss -> update parameters -> repeat (each full pass over data = **epoch**).
- Training ends when the loss stops falling or after a set number of epochs.
- Learning type decides the loss: labels (supervised), similarity (clustering), reward (RL).



Fig 1.12 : Under-fitting, good fit and over-fitting

1.14 Model Evaluation

Model evaluation = measuring how well the trained model works on **unseen data**. Training accuracy alone is misleading (over-fitting).

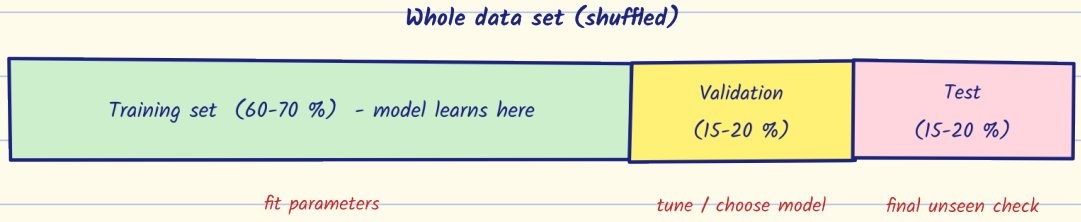


Fig 1.13 : Train / validation / test split

- **Training set** - learn parameters. **Validation set** - tune hyper-parameters, pick model. **Test set** - final honest score, used **once**.

- **k-fold cross-validation** : split into k parts; train on k-1, test on 1; repeat k times; average the score.
- **Over-fitting** : training error low, validation error high. **Under-fitting** : both errors high.

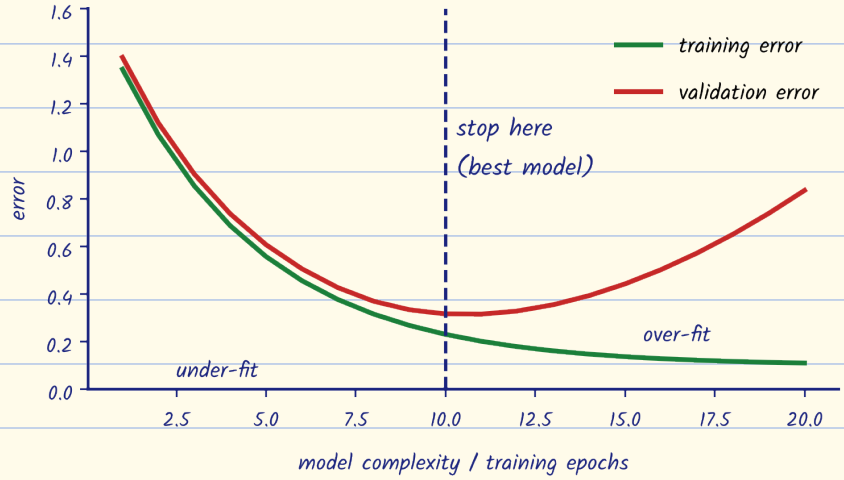


Fig 1.14 : Validation error rises after the best point (over-fitting)

Metric	Formula / meaning
Accuracy	(TP + TN) / total predictions
Precision	TP / (TP + FP) - of predicted positives, how many are right
Recall	TP / (TP + FN) - of real positives, how many found
F1 score	2 x P x R / (P + R)
MSE / RMSE	For regression : mean of (y - y_hat)^2, and its square root

TP, TN, FP, FN come from the **confusion matrix** (true / false positives and negatives).

1.15 Model Prediction

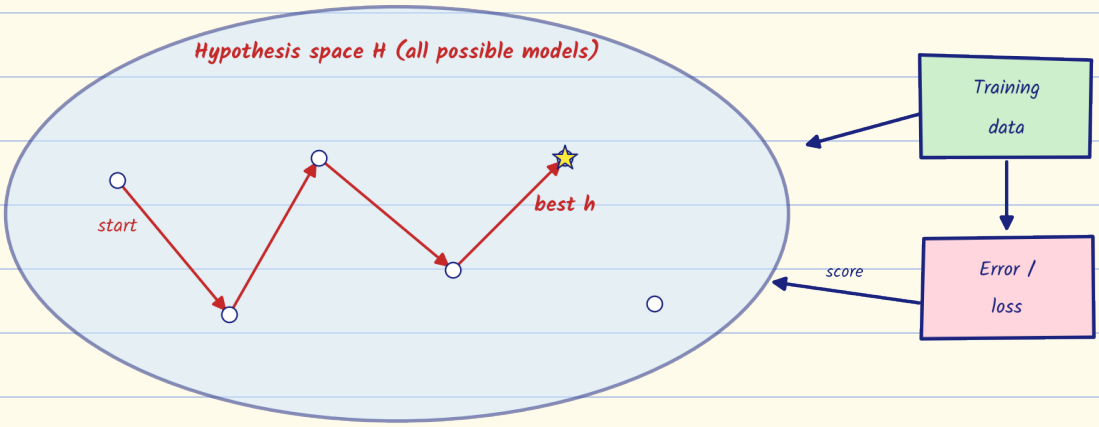
Model prediction (inference) = using the final trained model to give outputs for **new, real-world data**.

- The model is fixed now - the new input gets the **same pre-processing and features** as training data.
- Output: a class / probability (classification), a number (regression), a cluster id, or an action (RL).
- **Deployment:** app, web API, embedded device. Then **monitor** for **data drift** and **retrain** when accuracy falls.
- Example: trained spam model receives a new email -> features -> **P(spam) = 0.93** -> moved to spam folder.

Part 4 : Search and Learning, Data Sets

1.16 Search and Learning

Learning can be seen as a **search problem**: look through a huge **hypothesis space H** (all models we could build) to find the hypothesis **h** that best fits the training data.



learning = searching H for the h with the least error

Fig 1.15 : Learning as search in the hypothesis space

- **Hypothesis space H** - the set of all functions the learner can represent (e.g. all straight lines, all trees of depth 3).
- **Search strategy** : how we move in H - **gradient descent** (continuous), greedy tree building, **genetic algorithms**, hill climbing, exhaustive search (too costly).
- **Evaluation function** guides the search: error / loss on training data (or likelihood).
- **Version space** : the set of all hypotheses consistent with the examples; shrinks as examples increase (Find-S, Candidate-Elimination).
- **Inductive bias** limits H or orders it, so search is possible and generalises.
- Too big an H -> over-fitting and heavy search; too small -> may not contain a good h.

1.17 Data Sets

A **data set** = a collection of samples (rows) with their features (columns) used to train and test models. Standard **benchmark data sets** let us compare algorithms.

Data set	Type / task	Size and details
Iris	Numeric, classification	150 flowers, 4 features (sepal / petal length, width), 3 classes

Data set	Type / task	Size and details
MNIST	Images, classification	70,000 handwritten digits 28x28, 10 classes (60k train, 10k test)
Boston / California housing	Numeric, regression	House features -> price
Titanic	Mixed, binary classification	Survived or not; age, sex, class
CIFAR-10 / ImageNet	Images, classification	Object photos; ImageNet has 1000+ classes
IMDB reviews	Text, sentiment	Positive / negative movie reviews
Wine, Breast cancer (UCI)	Numeric, classification	Chemical / cell measurements

- Sources : **UCI ML Repository**, **Kaggle**, scikit-learn built-in sets, OpenML.
- A good data set is **large, clean, representative and balanced**.
- Always split into **train / validation / test**. Use **stratified** split if classes are imbalanced.

* **Remember:**
Training data teaches the model, **validation data** tunes it, **test data** judges it. Never let test data leak into training.

Part 5 : Quick Revision and Exam Questions

Quick Revision

- **ML** = learning rules from data to improve on a task (Mitchell : T, E, P). AI > ML > DL.
- **Paradigms** : supervised (labels), unsupervised (no labels), semi-supervised (few labels), reinforcement (reward).
- **Rote** = memorise, no generalisation. **Induction** = examples -> general rule.
- **RL** : agent, environment, state, action, reward, policy; trial and error.
- **Data types** : numeric, categorical, ordinal, binary, text, image, time-series.
- **Matching** = compare new pattern with prototypes using distance / similarity.
- **Stages** : data acquisition -> cleaning -> feature engineering -> model selection -> learning -> evaluation -> prediction.
- **Feature selection** keeps a subset; **extraction** builds new features (PCA).
- **Representation** : feature vector, matrix X (n x d), label vector y, one-hot.
- **Learning** minimises loss; **evaluation** uses unseen test data; over-fit = low train, high test error.

- **Learning = search** in hypothesis space guided by error.

Likely exam questions

Short answer (2–3 marks)

1. Define machine learning (Tom Mitchell). How is it different from traditional programming?
2. Differentiate rote learning and learning by induction.
3. List four types of data with examples.
4. What is feature engineering? Give two techniques.
5. What are over-fitting and under-fitting?
6. Define hypothesis space and version space.

Long answer (8–10 marks)

1. Explain the paradigms of machine learning with suitable diagrams and examples.
2. Explain reinforcement learning with the agent-environment model.
3. Describe the various stages in a machine learning project with a neat block diagram.
4. Explain data acquisition, feature engineering and data representation in detail.
5. Explain model selection, learning, evaluation and prediction. Describe train / validation / test split and metrics.
6. Explain how learning can be viewed as a search problem. Mention matching and common data sets.